第 8 章 QEMU 虚拟硬件平台与设备驱动

初赛阶段在两种 QEMU 虚拟机上运行：RISC-V64（`qemu-system-riscv64 -machine virt`）和 LoongArch64（`qemu-system-loongarch64`）。两种架构的页表格式、异常编码、中断控制器和设备总线完全不同。把这些差异直接暴露给上层，调度器、内存管理器和系统调用处理代码就会散布架构条件分支。本章说明 Moss 如何通过三层平台抽象将差异封死在平台实现层，VirtIO 则在两种架构上提供行为一致的设备接口。

8.1 平台抽象层架构

8.1.1 三层隔离设计

Moss 将硬件差异分三层处理，每层只吸收它自己那一层的差异。

第一层是 `axcpu`，处理 CPU 本身的寄存器和异常机制。`axcpu/src/riscv/` 实现 RISC-V 的 Trap 上下文保存与恢复、用户态进入与返回；`axcpu/src/loongarch64/` 做同样的事，用 LoongArch 的 `era`/`badv` 等 CSR。两边向上提供相同的 `TrapFrame`、`context_switch`、`uspace` 接口。

第二层是 `axplat`，用 Rust trait 定义平台接口：`IrqIf`（中断注册与分发）、`TimeIf`（时钟读取与单次定时器）、`MemIf`（物理内存区域描述）、`ConsoleIf`（串口输出）。`axplat-riscv64-qemu-virt` 和 `axplat-loongarch64-qemu-virt` 各自实现这些 trait，通过 `#[impl_plat_interface]` 宏完成编译期静态绑定。

第三层是 `axhal`，把 `axplat` trait 调用包装成普通函数，供内核其余部分使用。调度器、内存管理器和驱动框架调用的是 `axhal::time::wall_time()`、`axhal::irq::register()` 这类接口，不直接触碰任何 ISA 特有寄存器。

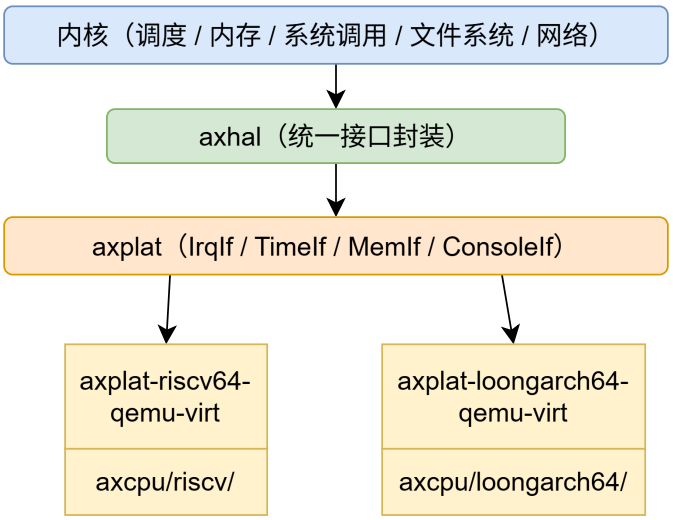


图 8-1　平台抽象三层结构

8.1.2 中断控制器抽象

RISC-V64 的外部设备中断经 PLIC 路由。`scause` 区分定时器（`S_TIMER`）、核间中断（`S_SOFT`）、外部中断（`S_EXT`）三类。处理外部中断时需先向 PLIC claim 获得设备 IRQ 号，处理完后再 complete 确认。

LoongArch64 的路由链不同：设备中断由 PCH-PIC 收集后送入 EIOINTC，再由 EIOINTC 向处理器核心报告；定时器中断通过 `ecfg` 寄存器的 `LineBasedInterrupt::TIMER` 位独立控制。

两套机制都实现了同一个 `IrqIf` trait：

方法	RISC-V64 实现	LoongArch64 实现
`set_enable`	操作 `sie` CSR 位或 PLIC priority/enable 寄存器	操作 `ecfg.lie` 位或 EIOINTC 使能寄存器
`register`	按 `scause` 值分别注册 timer / IPI / PLIC handler	按 `TIMER_IRQ` / `EIOINTC_IRQ` 分类注册
`handle`	外部中断时 claim PLIC 得到真实 IRQ 号，再调度 handler	读 EIOINTC 状态位后调度 handler
`send_ipi`	通过 SBI `send_ipi()` 指定 hart mask 发送	通过 EIOINTC IPI 功能发送

表 8-1　IrqIf 在两种架构下的实现：接口相同，硬件路径不同

驱动框架调用 `axplat::irq::register(irq, handler)` 时，对 PLIC 还是 EIOINTC 一无所知。

8.1.3 时钟接口抽象

RISC-V64 通过 SBI（`sbi_time::set_timer`）设置定时器，`time::read64()` 读取当前计数；LoongArch64 使用 CPU 内置的稳频计数器，`tcfg`/`tval` 寄存器控制定时器触发。两者同样通过 `TimeIf` trait 统一：`current_ticks()` 读各自的硬件计数器，`ticks_to_nanos()` 完成频率转换，`set_oneshot_timer()` 设置下一次中断。调度器的时间片计算和 `clock_gettime` 系统调用只操作 `axhal::time::monotonic_time()`，不感知底层计数器差异。

8.2 双架构 QEMU 平台对比

资源	RISC-V64 QEMU virt	LoongArch64 QEMU virt
启动方式	OpenSBI，ELF 内核直接加载	UEFI BIOS，ELF 内核直接加载
页表格式	Sv39 三级页表，`satp` CSR	LoongArch 四级页表，`PGDL`/`PGDH` CSR（见第 4 章）
中断控制器	PLIC，MMIO `0x0c00_0000`	PCH-PIC + EIOINTC
定时器	SBI timer，`mtime`/`mtimecmp`	稳频计数器，`tcfg`/`tval`
设备总线	VirtIO MMIO（固定地址范围枚举）	VirtIO over PCIe（ECAM + BAR 配置）
串口	8250 UART，MMIO	16550 UART，MMIO

表 8-2　两种 QEMU 平台的关键硬件资源对比

页表格式的差异已在第 4 章展开，本章不重复。

8.3 VirtIO 设备驱动

8.3.1 两条发现路径，一个驱动入口

赛方两种平台均提供两块 VirtIO 磁盘（`x0` 承载文件系统，`x1` 为额外测试磁盘）和一张 VirtIO 网卡，区别仅在于设备挂载的总线不同。

RISC-V64 上，QEMU virt 机型将 VirtIO 设备寄存器映射到固定 MMIO 地址范围。`axdriver` 初始化时遍历该范围，将物理地址转换为内核虚拟地址后调用 `probe_mmio_device()` 探测设备类型。

LoongArch64 上，VirtIO 设备挂在 PCIe 总线上。`axdriver` 通过 PCI ECAM 枚举 BDF，匹配厂商 ID `0x1af4`，按设备 ID 区分块设备（`0x1001/0x1042`）和网卡（`0x1000/0x1041`），配置 BAR 后调用 `probe_pci_device()` 初始化传输层。

两条路径都以 `VirtIoTransport` 为参数进入 `D::try_new(transport, irq?)`，其中 `D` 是实现了 `VirtIoDevMeta` 的具体类型（`VirtIoBlk` 或 `VirtIoNet`）。成功初始化的设备通过 `AxDeviceEnum` 注册到全局设备表，文件系统挂载和网络协议栈直接取用，不感知底层总线差异。

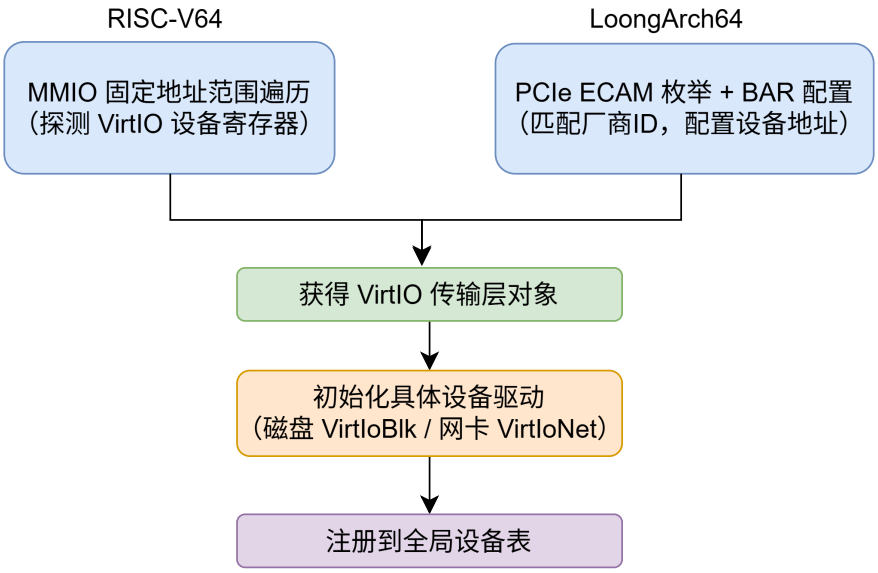


图 8-2　VirtIO 设备发现路径

8.3.2　VirtIO 队列与 DMA

VirtIO 协议以 virtqueue 为数据通道：驱动将控制信息和数据缓冲区组织为描述符链，写入可用队列（available ring）并通知 QEMU；QEMU 完成后将结果写入已用队列（used ring），驱动据此回收描述符。块设备用它提交扇区读写，网卡用它收发报文，两者共用同一套队列管理模型。

VirtIO 描述符中的缓冲区地址必须是物理地址。`VirtIoHalImpl` 实现 `VirtIoHal` trait，负责 DMA 页面的分配与地址转换：

```
// axdriver/src/virtio.rs
unsafe impl VirtIoHal for VirtIoHalImpl {
    fn dma_alloc(pages: usize, _dir: BufferDirection) -> (PhysAddr,
NonNull<u8>) {
        let Ok(vaddr) = global_allocator().alloc_pages(pages, 0x1000,
UsageKind::Dma)
        else { return (0, NonNull::dangling()) };
        let paddr = virt_to_phys(vaddr.into());
        (paddr.as_usize(), NonNull::new(vaddr as _).unwrap())
    }
    unsafe fn dma_dealloc(_paddr: PhysAddr, vaddr: NonNull<u8>, pages: usize) -
> i32 {
```

```
        global_allocator().dealloc_pages(vaddr.as_ptr() as usize, pages,
UsageKind::Dma);
        0
    }
    unsafe fn mmio_phys_to_virt(paddr: PhysAddr, _size: usize) -> NonNull<u8> {
        NonNull::new(phys_to_virt(paddr.into()).as_mut_ptr()).unwrap()
    }
}
```

代码 8-1　`VirtIoHalImpl` 的 DMA 分配与地址转换，两种架构共用同一份实现

`dma_alloc` 从全局分配器申请物理连续内存并将物理地址填入描述符；请求结束后 `dma_dealloc` 释放这些页面，避免设备访问已回收的缓冲区。MMIO 寄存器通过 `mmio_phys_to_virt` 映射后访问。这套接口在 RISC-V64 和 LoongArch64 上完全相同。

8.4　跨平台行为一致性

axplat trait 是 ISA 差异的终止线。内核核心模块只调 `axhal` 的接口，不直接操作 PLIC 或 EIOINTC，不读取 `mtime` 或 LoongArch 稳频计数器，也不关心 VirtIO 设备在 MMIO 还是 PCIe 总线上。平台实现层把硬件差异翻译成统一接口，差异到此为止，不再向上传递。

赛方为两种架构提供相同的设备拓扑，内核在两种平台上经历相同的初始化序列和 I/O 路径。用户态进程看到的文件系统语义、网络行为和时钟语义不因底层 ISA 而不同。